

Availability Is Not Enough: When Symbolic Tools Help—and Hurt—LLMs on Planning Tasks

Anonymous submission

Abstract

Sound symbolic planners and validators are correct by construction, and language models are increasingly able to call them as tools — but does access actually help an LLM on planning tasks, and when? Using PDDL as a setting where a deterministic oracle can grade every answer exactly, we evaluate five single-tool-use tasks (solving, domain, problem, and plan validation, and state-trajectory simulation) across five open-weight models, scoring strictly against the solver/validator rather than by self-report. A three-arm design separates tool *availability* from a one-sentence *steering* nudge, every proportion carries a confidence interval, and a signed-significance rule distinguishes help from harm. The benefit is strongly regime-dependent: two tasks essentially cannot be done without the tool, two are improved over weak baselines (one rescued from below its trivial majority guess), and one — plan checking — exposes the general lesson. There, for one of three ≥ 9 B models, merely making the tool *available* sharply *lowers* success (a 67 pp drop), because the model reasons in prose instead of calling the validator; the failure is silence, not error (it answers correctly $>99\%$ of the times it does call), and one steering sentence reverses it. Measured in tokens, the tool pays for itself precisely where the model cannot do the task unaided. The pattern holds under reasoning mode and an anonymized-domain contamination control. The bottleneck throughout is invocation *propensity* — a volatile, model- and prompt-dependent behavior — not capability or tool accuracy.

Introduction

Large language models are increasingly deployed as agents that call external tools, and automated planning is an unusually clean setting in which to ask whether that helps. Decades of research have produced planners and validators that are *correct by construction*: a plan a sound solver returns provably reaches the goal, and a verdict a sound validator issues is sound (Helmert 2006; Howey, Long, and Fox 2004). If a language model can simply *call* such a tool, it should be able to offload exactly the parts of a PDDL task — search, state tracking, plan checking — on which language models are known to be unreliable (Valmeekam et al. 2023; Kambhampati et al. 2024). Whether, and *when*, this actually improves

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

end-to-end performance — and at what cost — is the question we study.

The question is less settled than it may appear. One line of work tests LLMs *as* planners and finds they fall well short of solvers (Valmeekam et al. 2023); a second uses LLMs as *formalizers* that translate language into PDDL for a downstream solver (Liu et al. 2023; Tantakoun, Muise, and Zhu 2025); and a third measures tool-use competence in isolation, without a planning oracle (Patil et al. 2025). The specific question — does access to a sound PDDL tool improve a model across the planning task pipeline, measured end-to-end — was raised by the PDDL Planning Copilot (Benyamin et al. 2025), but its evaluation was preliminary: a limited prompt set, success scored by tool *selection* rather than end-to-end correctness, no confidence intervals, and no control for memorization of public benchmark domains. A clean answer requires controlling for several confounds at once — whether the model is even prompted to use the tool, whether the baseline is flooded or already competent, reasoning-mode decode budgets, and contamination — which is what this paper sets out to do.

We evaluate five single-tool-use PDDL tasks (*solve*, *validate_domain*, *validate_problem*, *validate_plan*, *simulate*) across five open-weight models, grading every response strictly against a deterministic solver/validator oracle rather than by self-report. A three-arm design separates tool *availability* (the tool is present) from a one-sentence *steering* nudge (the model is told to use it), every proportion carries a confidence interval, and verdicts use a signed-significance rule that distinguishes a tool that helps from one that hurts. A separate sweep over anonymized domains controls for contamination, and a token cost-of-pass analysis asks what the tool is worth per correct answer.

The benefit turns out to be strongly regime-dependent rather than uniform. Of the five tasks, two (*solve*, *simulate*) essentially do not happen without the tool; on two (*validate_domain*, *validate_problem*) the tool improves over weak baselines — one genuinely partial, the other at or below the trivial majority guess; and one (*validate_plan*) is a cautionary case in which merely making the tool available can *lower* success — for one of three ≥ 9 B models, by 67 pp, as it reasons in prose instead of calling the validator — while a single steering sentence repairs it. Where the unaided baseline has headroom, the tool’s advantage grows with plan length; it is cost-effective in tokens exactly where the model cannot do the

task unaided; and the pattern survives reasoning mode and an anonymized-domain control. Concretely, we contribute:

- A controlled three-arm, five-task, five-model evaluation of symbolic PDDL tool utility with strict end-to-end grading, confidence intervals, and a signed-significance verdict rule that separates help from harm.
- The finding that tool utility is regime-dependent, and the identification of *invocation propensity* as a large, volatile, prompt- and mode-dependent free variable — not capability or tool accuracy — as the binding constraint: success factors as $P(\text{call}) \times P(\text{correct} \mid \text{call})$, with essentially all between-arm, between-model, and cross-mode variance living in the first factor. This yields the counterintuitive result that, for a model that under-calls, tool *availability* can hurt a task it already does well, via a silence-not-error mechanism that one steering sentence reverses.
- A token cost-of-pass analysis showing the tool pays for itself precisely where the unaided baseline is floored, and is a luxury where it is already strong.
- Robustness evidence: the verdict pattern reproduces under reasoning mode through budget-insensitive statistics, and an anonymized-domain contamination probe returns null.

Background and Related Work

Automated planning studies how to synthesize a course of action that transforms an initial state into one satisfying a goal. The Planning Domain Definition Language (PDDL) (McDermott et al. 1998; Fox and Long 2003) is the field’s standard formalism, and decades of research have produced sound, complete solvers such as Fast Downward (Helmert 2006) and FF (Hoffmann and Nebel 2001) for classical planning and ENHSP (Scala et al. 2016) for numeric planning, together with plan validators such as VAL (Howey, Long, and Fox 2004). These systems are correct by construction: a plan they return provably reaches the goal, and a verdict they issue is sound. They are, however, brittle to the formal-modelling step — a domain or problem must be expressed in precise PDDL before any solver can run.

Large Language Models for Planning

Large language models have been probed extensively as planners. Systematic benchmarks show that, prompted to produce plans directly, even strong LLMs fall well short of solver-level reliability (Valmeekam et al. 2023). Kambhampati et al. (2024) argue from this evidence that autoregressive LLMs cannot plan or self-verify on their own, and propose *LLM-Modulo* frameworks in which the model is paired with external, model-based verifiers that supply the soundness it lacks. The present study is empirical evidence for exactly this division of labour: we measure what an LLM gains when a sound planner or validator is available as a callable tool, rather than asking the model to be the planner.

Large Language Models as Planning Formalizers

A complementary line treats the LLM not as the planner but as a *formalizer* that translates natural language into PDDL,

delegating search to a sound solver. LLM+P (Liu et al. 2023) pipelines an LLM into a classical planner; Guan et al. (2023) use LLMs to construct world models for model-based planning; Plansformer (Pallagani et al. 2022) fine-tunes for plan generation; and pretrained LLMs have been applied to generalized planning in PDDL domains (Silver et al. 2024). Tantakoun, Muise, and Zhu (2025) survey this rapidly growing “LLMs-as-formalizers” area, and Huang and Zhang (2025) characterize its limits, showing that although models are largely robust to lexical perturbation of domain symbols, formalization quality degrades as natural-language descriptions move away from templated phrasing. This literature largely studies the *generation* of PDDL, evaluated on a single downstream question (does the produced model solve?). We instead evaluate LLMs *using* planning and validation tools across five distinct PDDL tasks, with strict end-to-end grading against a solver oracle.

Tool Use, Agent Protocols, and Their Evaluation

Equipping LLMs with external tools is now standard, from learned tool invocation (Schick et al. 2023) to large API repositories (Qin et al. 2024). The Model Context Protocol (MCP) (Anthropic 2024) standardizes how tools are described and called, and is the mechanism by which we expose planners and validators to the model. A growing body of benchmarks measures tool-use competence — single-turn function calling (Patil et al. 2025) and multi-turn tool-agent-user interaction (Yao et al. 2024), among others. Our setting sits deliberately at the simple end of this spectrum: each task admits exactly one relevant, correct-by-construction tool, so the appropriate analog is single-call function selection rather than long-horizon orchestration — a point we use to calibrate our results.

Positioning

The system most directly related to this study is the PDDL Planning Copilot (Benyamin et al. 2025), which wraps planners and validators behind an MCP interface and a single natural-language entry point. Its original evaluation was preliminary — a limited prompt set, no confidence intervals, and no control for memorization of public benchmark domains. This paper contributes a substantially more rigorous evaluation of whether such symbolic tools help LLMs: five single-tool-use PDDL tasks across five models, a three-arm design that separates tool *availability* from a one-sentence steering *nudge*, confidence intervals with signed significance, a contamination control over anonymized domains, and a token-cost (cost-of-pass) analysis. Cross-benchmark generalization (e.g., PlanBench (Valmeekam et al. 2023)) and published formalizer baselines (Huang and Zhang 2025) are deliberately out of scope here (see Future Work).

Methodology

We ask a single, deliberately narrow question: *when a sound symbolic planner or validator is available as a callable tool, how much does it improve a language model’s performance on individual PDDL tasks, and at what token cost?* The study crosses five PDDL tasks, five open-weight

Task	Graded correct iff (vs. deterministic oracle)
solve	returned plan is certified valid (goal reached, all preconditions hold)
validate_domain	VALID/INVALID verdict matches the oracle
validate_problem	verdict matches the oracle
validate_plan	verdict matches the oracle
simulate	full state trajectory equals the oracle’s (canonical form; no partial credit)

Table 1: The five single-tool-use tasks and their strict success criteria. Each task has a single relevant, correct-by-construction tool per instance (*solve* selects the classical or numeric planner).

models, two reasoning modes, and three experimental arms, and grades every response strictly against a deterministic solver/validator oracle rather than by self-report. The harness, fixtures, and locked findings reproduce and substantially strengthen the preliminary evaluation of the PDDL Planning Copilot (Benyamin et al. 2025), adding end-to-end grading, confidence intervals, a steering control, and a contamination probe.

Tasks, Oracle, and Fixtures

Each model is given five PDDL tasks, one tool-relevant per task: *solve* (find a plan reaching the goal), *validate_domain* and *validate_problem* (judge whether a domain or problem file is well formed), *validate_plan* (judge whether a given plan is correct), and *simulate* (produce the full state trajectory a plan induces, action by action). Before any model is run, an oracle generates ground truth by calling the symbolic tools directly — Fast Downward (Helmert 2006) and ENHSP (Scala et al. 2016) for planning, and a sound, pure-Python PDDL validator (built on the `unified-planning` library) for the verdict and trajectory tasks. The same deterministic tools both define ground truth and, in the tool arms, are what the model may call, so a correctly invoked tool is correct by construction; what the experiment measures is therefore whether and how the model invokes the tool, not the tool’s own accuracy. Grading is end-to-end and unforgiving (Table 1): a *solve* answer counts only if the returned plan is independently certified valid; a *simulate* answer counts only if its entire trajectory equals the oracle’s under a canonical normal form (sorted, case-folded, with the boolean/numeric fluent shapes unified), with no partial credit and no free-text fallback.

The fixture corpus is 20 PDDL domains — 10 classical (e.g., blocksworld, gripper, rovers, satellite, depots) and 10 numeric (e.g., counters, farmland, sailing, drone), drawn from the earlier study’s released dataset (Benyamin et al. 2025) and public benchmark suites — each shipping matched valid and mutated-invalid files so the verdict tasks score against both positive and negative instances (balanced one-to-one for *validate_problem* and *validate_plan*; *validate_domain* carries a 5:1 positive-to-negative ratio, which widens its negative-arm interval). Every committed positive

is re-validated and every negative is confirmed rejected at startup, so a fixture that drifts from its label aborts the run. Holding model, reasoning mode, and arm fixed, one cell comprises 4,560 graded trials: *solve* 300, *validate_domain* 360, *validate_problem* 600, *validate_plan* 3,000, and *simulate* 300. Each cell spans three neutral paraphrases per task (imperative, declarative, interrogative framings), a linguistic-robustness probe; the steering directive introduced below is held byte-identical across the three, isolating steering variance from paraphrase variance.

Models and Serving

We evaluate five open-weight instruction-tuned models spanning over an order of magnitude in scale (0.8B–35B) and two families: Qwen3.5-0.8B, Qwen3.5-4B, Qwen3.5-9B, a Gemma mixture-of-experts model (≈ 26 B total / ≈ 4 B active parameters, “Gemma-MoE-26B”), and Qwen3.6-35B.¹ Headline conclusions are drawn from the three larger models (≥ 9 B: Qwen3.5-9B, Gemma-MoE-26B, Qwen3.6-35B); the 4B model is reported for scaling context and the 0.8B model isolates a small-model failure mode (Results). All models are served locally with vLLM on a single workstation-class GPU, decoding deterministically (temperature 0), so each trial is a single fixed sample and the confidence intervals we report quantify sampling uncertainty over the fixture instances (treated as the inferential population), not decoding stochasticity, which temperature 0 removes. The context window is 16,384 tokens; per-task decode caps are 8,192 tokens for *solve* and 6,144 for the others; tool-using runs may iterate up to ten tool calls. The reasoning mode is toggled per run (*think* = on/off via the serving runtime’s thinking switch); crucially, under *think* = on the reasoning trace and the final answer share the *same* per-task decode cap, a shared budget that drives the confound we treat explicitly below. Tools are exposed to the model through the Model Context Protocol (Anthropic 2024) as standard function definitions; system prompts are thin per-task policy stubs, with all argument schemas carried by the tool definitions rather than the prompt.

The Three-Arm Design

Every (model, mode, task) is run in three primary arms (a fourth, control arm follows); arms are scored separately and never pooled, and every comparison is between two arms of the *same* cell:

- **no-tools** — the model answers from parametric knowledge. The PDDL tools are removed, but per-task JSON-schema-constrained decoding remains, so the baseline is not crippled by formatting alone; only the symbolic tools are absent.
- **tool-available (plain)** — the *byte-identical* neutral task prompt, now with the tools exposed. (The thin per-task

¹Hugging Face checkpoints: Qwen/Qwen3.5-0.8B, Qwen/Qwen3.5-4B, and Qwen/Qwen3.5-9B, served in 16-bit precision; the two mixture-of-experts checkpoints, `cyankiwi/gemma-4-26B-A4B-it-AWQ-4bit` and `cyankiwi/Qwen3.6-35B-A3B-AWQ-4bit`, are community AWQ-INT4 quantizations.

system prompts are structural mirrors sharing the same role-framing first sentence and sentence count, and differ in their policy clause — “use the available tool” vs. “use your own reasoning,” the latter adding an output-format reminder on the verdict tasks — a small instruction-surface difference we revisit in Limitations.) Differencing this arm against no-tools yields the *availability gap*, which isolates tool presence.

- **tool-steered** — the neutral prompt plus exactly one appended sentence directing the model to use the tool. Differencing against the plain arm yields the *steering gap*.

The two contrasts map onto the function-calling literature: the plain arm measures *relevance detection* (does the model spontaneously invoke the right tool when its existence is known but not urged?) and the steered arm measures *function selection* (told to use a tool, does it pick the correct one?) in the sense of the Berkeley Function-Calling Leaderboard (Patil et al. 2025). Because the per-task tool split enforces each tool’s required arguments through its schema, argument-filling errors are largely structural, so the steered contrast isolates selection rather than parameter construction. A fourth, *control arm* — the steering directive with the tools still removed — tests whether the directive alone moves the no-tools floor; it is reported as a control, not a primary outcome. The *realizable benefit* of the tool, used as the cross-mode spine below, is $\text{success}(\text{tool-steered}) - \text{success}(\text{no-tools})$: what a deployment that actually prompts for tool use would realize.

Metrics, Confidence Intervals, and Signed Significance

The primary outcome is the strict end-to-end *success rate* of Table 1, aggregated over all instances and the three paraphrases of an arm. In the tool arms we additionally record *tool_selected* — whether the model called the task-matching tool at all; the gap between *tool_selected* and *success* quantifies “knows *what* to call but cannot call it correctly.” Failures are typed (tool not selected, wrong tool, verdict mismatch, unparseable output, trajectory mismatch), with cap-truncated responses re-tagged as truncations rather than masquerading as content failures. Every reported proportion carries a Wilson 95% score interval (Wilson 1927), and *arms are never pooled*: each statistic comes from exactly one (model, mode, arm) corpus, and only differences *between* such statistics are formed. Because each instance is scored under three paraphrases, trials within an instance are not independent: the intraclass correlation is ≈ 0.6 and the design effect ≈ 2.2 – 2.7 , so the effective sample size is roughly half the nominal count and these Wilson intervals are correspondingly anti-conservative. We verified that every headline verdict (+50–92 pp, and the smallest reported gaps) remains CI-disjoint after inflating intervals by $\sqrt{2.7}$, and flag the one verdict-relevant gap that does not (the -5 pp *validate_problem* steering dip) individually.

Verdicts use a *signed* significance rule. A per-model gap counts as favorable evidence only when the two arms’ 95% intervals are disjoint *and* the gap points the hypothesized way (the tool helps); a disjoint gap pointing the other way is tallied

separately as significant-*against* — a finding, not noise. A research question reads YES when favorable significant gaps dominate with none against, MIXED when significant against-gaps are present, and NO when no significant favorable movement exists. This sign-awareness is load-bearing: on *validate_plan* mere tool availability moves one $\geq 9B$ model by -67 pp (it stops calling the tool and returns no gradeable answer), a wildly significant gap that a sign-blind disjointness test would miscredit as the tool “having an effect”; the signed rule instead reads that task MIXED, which is what the data say. Requiring disjoint 95% intervals is deliberately conservative — it controls the false-positive rate well below $\alpha = 0.05$, at the cost of power on small gaps — so a no verdict means “no movement we are willing to call significant,” not “no movement.” We do not apply a family-wise correction across cells: the reported effects are large, and we treat the few borderline gaps as exploratory rather than confirmatory.

Cross-Mode Aggregation and the Robust Floor

think = off is the clean headline: the model answers directly, so the availability and steering gaps measure the tool, unfounded. Under *think* = on the shared decode budget truncates a large fraction of $\geq 9B$ no-tools trials (the baseline “reasons itself to death”), so raw *think* = on gaps conflate the tool’s value with baseline budget collapse. We therefore make cross-mode claims only through budget-insensitive statistics. The spine is the realizable benefit (steered, not plain: under *think* = on the plain arm largely stops calling the tool, so its availability gap also folds in a mode-dependent tool-calling failure that steering repairs). Gaps carry Newcombe MOVER intervals (Newcombe 1998), and the mode difference $\Delta(\text{on} - \text{off})$ carries a MOVER interval for the difference of two independent gaps — valid because the two reasoning-mode corpora share no trials. The *robust floor* is the minimum realizable benefit over the two reasoning modes (the smaller of the two mode point estimates, a deliberately conservative construction): a lower bound that neither mode’s decode budget can inflate. Against a deliberately conservative $+30$ pp threshold this sorts the tasks into *robust* (every $\geq 9B$ floor $\geq +30$ pp), *sole-source* (baseline at 0 in both modes — a benefit with no baseline cannot be budget-confounded), and *budget-dependent* (some floor below $+30$ pp, where the large *think* = on stretch is precisely the artifact).

Contamination Control

Because the domains are public, an unaided baseline could be inflated by memorization. A separate sweep re-ran the entire matrix on structurally identical domains with every surface symbol (domain, predicate, and object names) renamed. Under *think* = off the clean no-tools probe is near-null — mean $|\Delta(\text{canonical} - \text{anonymized})|$ success 1.1 pp (max 3.7) with zero CI-disjoint task cells — so where the baseline has headroom it carries no detectable memorization signal, and the availability gap (tool arm minus this clean baseline) therefore cannot be a contamination artifact on the baseline side. The only CI-disjoint cells appeared under *think* = on and trace to tokenization, not memory: the renamed prompts ran $\approx 5\%$ longer, so more trials truncated under the shared budget while success-given-completion was unchanged — itself

a live demonstration of the budget confound the cross-mode analysis controls for.

Results

Unless stated otherwise, every figure is *think* = off (the unconfounded read) on the three $\geq 9B$ models, with reasoning mode treated separately in the Robustness subsection. We report the strict end-to-end success rate of Table 1; the *availability* gap is $+tool(plain) - no-tools$ and the *steering* gap is $+tool(steered) - +tool(plain)$, each significant when the two arms’ 95% intervals are disjoint.

We organize the evaluation around six research questions — the first four asking, per task, whether and how the tool helps, the last two whether its advantage tracks two difficulty axes:

- RQ1.** On the two file-validation tasks (*validate_domain*, *validate_problem*), where the unaided model has some competence, does tool availability add value?
- RQ2.** On *solve*, can the unaided model find valid plans, and how much does the tool add?
- RQ3.** On *validate_plan*, a task the unaided model already does well, does mere availability help or hurt — and does steering change the answer?
- RQ4.** On *simulate*, can the unaided model produce an exact state trajectory at all?
- RQ5.** Does the tool’s advantage grow with plan length?
- RQ6.** Does it grow with object count?

Two regime vocabularies appear below and should not be conflated. A *task* axis sorts the five tasks by their unaided baseline: *sole-source* (no unaided success), *headroom* (a weak but improvable baseline), and *mixed* (availability can hurt). A separate *cross-mode* axis (Robustness) sorts tasks by whether the realizable benefit survives reasoning mode: *robust*, *budget-dependent*, and again *sole-source*. The two axes share only the word *sole-source* — a task with no baseline is *sole-source* either way — and are otherwise independent. Table 2 is the task-axis scorecard: two tasks (*solve*, *simulate*) essentially do not happen without the tool; two (*validate_domain*, *validate_problem*) are improved over weak baselines; and one (*validate_plan*) is an instructive mixed case in which mere availability can *hurt* and a one-sentence nudge repairs it.

Two Tasks the Model Cannot Do Without the Tool

Solve and *simulate* are *sole-source*: the tool is not an improvement but a precondition (Figure 1). Unaided *solve* is floored at 8–11% across the $\geq 9B$ models; the tool lifts the plain arm to 63–99% and the steered arm to 92–100%, an availability gap of +54–92 pp that is CI-disjoint and favorable for all three (RQ2 YES). Where the plain arm leaves headroom — Qwen3.6-35B invokes the planner on only 68% of trials and reaches 63% — steering raises invocation to 94% and success to 92% (+29 pp). The bottleneck is invocation, not the planner: accuracy-when-calling is $\geq 93\%$ for every $\geq 9B$ model.

Simulate is starker: unaided success is 0% — for every model, in both reasoning modes, across all 3,000 trials (a

rule-of-three 95% upper bound of $\leq 0.1\%$). The tool reaches 65–92% (plain) and 83–97% (steered). The zero is not a grading artifact. Success demands canonical-form deep-equality of the *entire* state trajectory against the oracle (structured JSON, no partial credit, no free-text fallback), and the unaided failures decompose ($\geq 9B$) into 68% unparseable trajectory JSON, 29% truncated at the 6,144-token cap, and only 3% parsed-but-wrong (Figure 2): hand-tracking a multi-step trajectory is simply outside these models’ reach, independent of output formatting.

Two Validation Tasks: Headroom and Rescue

On *validate_problem* the unaided model is genuinely partly competent: the set is balanced one-to-one, so a constant “VALID” guess scores $\approx 50\%$, and the $\geq 9B$ models clear that floor before any tool. The availability gap is +21–30 pp (3/3 favorable-significant, 0/3 against): the tool adds real value over a non-trivial baseline (RQ1 YES). *Validate_domain* is different, and instructive. Because its fixtures run 5:1 valid-to-invalid, a constant “VALID” already scores 83%; yet unaided, every $\geq 9B$ model lands *at or below* that trivial line (pooled 26–78%) and discrimination is unreliable across the board, from near-chance to only modest (balanced accuracy 53–74%; Table 5). Qwen3.5-9B is the clearest case — it answers “INVALID” almost reflexively (true-positive rate 12%, true-negative rate 95%), scoring 26% pooled but 53% balanced. The tool does not *refine* a competent baseline here; it *rescues* a task the models cannot do reliably, lifting balanced accuracy to 95–100%. The verdict holds from 4B up; the 0.8B model reverses on *validate_problem* (–25 pp), an instance of small-model tool-mishandling — it calls the right tool but omits required arguments. Steering does not significantly help either validation task (–5–+1 pp); the one CI-disjoint movement — a –5 pp dip on *validate_problem* for the 9B — points mildly against and does not survive the paraphrase-clustering correction above, so we treat it as non-confirmatory. Once a capable model has the tool it already calls it, and the nudge matters only where the plain arm under-calls — which is exactly the next case.

When Availability Backfires, and Steering Repairs It

Validate_plan is the one task the unaided model already does well ($\geq 9B$ no-tools 80–91%), and it is the most informative. Merely making the tool available is significant-*against* for two of three $\geq 9B$ models: Gemma-MoE-26B falls 88 → 21% (–67 pp) and Qwen3.6-35B 91 → 82% (–9 pp); only Qwen3.5-9B improves (80 → 93%, +13 pp). One favorable, two against ⇒ MIXED — and only because the verdict counts direction: a sign-blind test would read Gemma’s wildly CI-disjoint –67 pp as the tool “having an effect” and miscall the task YES.

The collapse is *silence, not error* (Figures 3 and 2): under the plain prompt Gemma-MoE reasons in prose instead of calling the validator, so the –67 pp is a tool-calling failure, not a verdict failure. Steering repairs it — the steered arm is favorable-significant for all three $\geq 9B$ models (+3–72 pp) — so the tool’s value on a task the model can already do is

RQ / task	verdict	evidence ($\geq 9B$, <i>think=off</i>)
RQ1 <i>validate_domain</i> / <i>validate_problem</i>	YES	<i>validate_problem</i> availability +21–30 pp over a balanced $\approx 50\%$ floor (3/3 favorable, 0/3 against); <i>validate_domain</i> rescued from near-chance discrimination (balanced acc. 53–74%) to 95–100%
RQ2 <i>solve</i>	YES	unaided floored at 8–11%, tool 63–99%; availability +54–92 pp
RQ3 <i>validate_plan</i>	MIXED	availability –67–+13 pp (1/3 favorable, 2/3 against); steering repairs (+3–72 pp)
RQ4 <i>simulate</i>	YES	unaided 0%, tool 65–92%; availability +65–92 pp
RQ5 plan length	YES	<i>validate_plan</i> advantage widens +5 $\rightarrow$ +7 $\rightarrow$ +27 pp with reference length
RQ6 object count	NO	<i>validate_problem</i> advantage flat (+22/+23/+23 pp) across object-count bins

Table 2: Scorecard: six research questions, six verdicts. A gap counts as favorable evidence only if the two arms’ 95% intervals are disjoint *and* it points the hypothesized way; disjoint gaps pointing the other way are tallied as significant-against (Methodology).

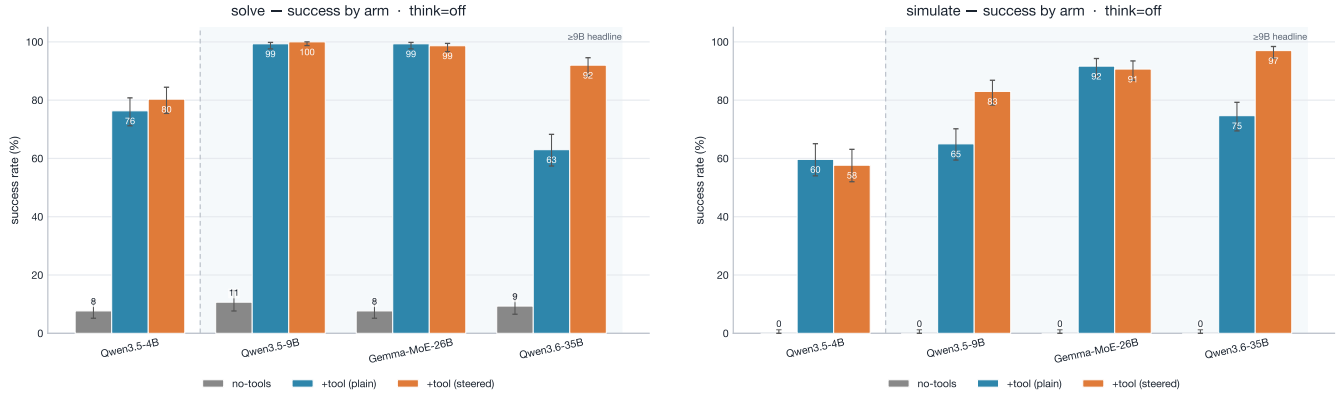


Figure 1: Sole-source tasks (success by arm, *think=off*; whiskers are Wilson 95% CIs; the $\geq 9B$ headline set is shaded). *solve* (left) is floored at 8–11% unaided; *simulate* (right) is 0% for every model. The tool lifts both decisively, and steering adds a further lift wherever the plain arm under-calls (e.g. Qwen3.6-35B on *solve*).

real but gated on invocation, and a single directive sentence unlocks it.

The decomposition makes the mechanism exact. Writing strict success as $P(\text{call}) \times P(\text{correct} \mid \text{call})$, Gemma-MoE’s plain-arm collapse is entirely in the first factor: it calls the validator on only 21% of trials, but *when it calls it* is 99% correct, and on the 79% where it stays silent it is 0% correct (a missing tool call is an ungradeable answer, never a lucky guess). Steering moves only $P(\text{call})$ — 21 $\rightarrow$ 94% — and success follows it almost exactly ($0.94 \times 0.99 = 0.93$). Across every $\geq 9B$ tool cell except 9B *simulate* ($\approx 81\%$), $P(\text{correct} \mid \text{call})$ stays $\geq 92\%$ and varies little between arms (Table 4); essentially all between-arm and between-model variance in success is variance in *whether the model calls*. The tool’s accuracy is not in question — its invocation is.

This also dissolves an apparent size inversion. In the plain arm the 9B “beats” the 35B (*solve* 99% vs. 63%; *validate_plan* 93% vs. 82%), but this is tool-call *propensity*, not capability: success tracks tool-use almost 1:1, accuracy-when-calling is $\geq 93\%$ for both, and steering closes the gap (35B *solve* 63 $\rightarrow$ 92%). The propensity even flips with reasoning mode (under *think=on* the 35B calls the validator on 99.9% of trials, the 9B on 69%). The 35B’s unaided baselines exceed the 9B’s on every validation task; the inversion is a behavioral difference in spontaneous tool adoption, not a capability gap. Because our two largest models are AWQ-INT4

while the 9B is 16-bit, we read this cross-model inversion as a propensity phenomenon we observe rather than a controlled size effect (see Limitations).

Does the Advantage Track Difficulty?

An advantage can only be *seen* to change where the no-tools baseline has room to move, which makes *validate_plan* the diagnostic case (RQ5 YES): as the reference plan lengthens, unaided accuracy degrades (94 $\rightarrow$ 91 $\rightarrow$ 62% over short/medium/long bins) while the tool holds ($\approx 99 \rightarrow 98 \rightarrow 89\%$), so the gap widens +5 $\rightarrow$ +7 $\rightarrow$ +27 pp — the harder the instance, the more the tool is worth. The other two length-sensitive tasks bracket this case: *solve*’s gap is large but roughly constant (+87 $\rightarrow$ +87 $\rightarrow$ +89 pp), both arms pinned (baseline floored, tool near ceiling); *simulate*’s gap is large but *declines* (+100 $\rightarrow$ +93 $\rightarrow$ +77 pp, CI-disjoint), because although its baseline stays floored at 0, the tool arm itself degrades on long trajectories — the one place tool-assisted success erodes with difficulty. Object count, by contrast, moves nothing: the *validate_problem* gap is flat at +22/+23/+23 pp and the other tasks are flat too (RQ6 NO). Plan length is a difficulty axis the tool’s advantage tracks; object count is not.



Figure 2: How models fail: per-trial outcome composition by task and arm ($\geq 9B$, *think=off*, pooled), each bar normalized to 100%. Every trial is one of *success* or a single failure type, separating budget *truncation* from *no tool call* (silence) from genuine content errors (*unparseable*, *wrong content*, *tool-call error*). The figure substantiates the failure-mode claims made throughout: unaided *simulate* is 68% unparseable + 29% truncated; unaided *solve* is dominated by unparseable output; the unaided validation baselines fail by *wrong content* (a parseable but incorrect verdict); and on *validate_plan* the plain tool arm carries a large *no tool call* band — the silence that steering removes.

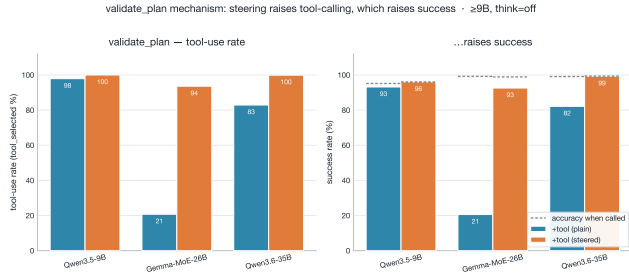


Figure 3: *validate_plan* mechanism ($\geq 9B$, *think=off*). Left: tool-use rate (plain vs. steered); right: success. Where the plain arm under-calls the validator (Gemma-MoE), steering restores invocation, and success rises with it — the tool’s value is gated on the model actually calling it.

What the Tool Costs

The tool is not free (Figure 4). Per trial it raises total token consumption (input + output, summed over turns) several-fold — $\approx 4\text{--}6\times$ on the $\geq 9B$ models, and up to $\approx 15\times$ for the smaller 4B — and the cost is *input*, not output: the unaided model is output-heavy (input:output $\leq 1:1$, as low as 0.3:1 on *solve* — it reasons in the open), while the tool arms are strongly input-heavy ($\approx 6\text{--}12:1$) because the tool schemas and tool outputs are re-fed every turn. (Server-side prefix caching, $\approx 90\%$ on this roster, makes the re-fed *schemas* cheap to compute, but tool *outputs* are uncached and the tokens are still real context.)

Whether it is worth paying depends entirely on the baseline — the choice of metric flips the verdict by $\approx 10\times$ on identical data, so we report *cost-of-pass* = total tokens $\div$ successful trials (lower is better), which factors exactly as (tokens-per-trial ratio) $\div$ (success-rate ratio). Where the baseline is floored the tool pays for itself: on *solve* it is $\approx 0.3\text{--}0.4\times$ the

cost per success (e.g. $\approx 4\times$ more tokens $\div \approx 9\text{--}13\times$ more correct answers), and on *simulate* it is the *only* producer of any success (no-tools cost-of-pass is infinite). Where the baseline is already strong the tool is a luxury: $\approx 3\text{--}5\times$ costlier per success on most *validate_** cells ($\geq 9B$), where it is only modestly more often right and the token premium dominates. The honest headline is therefore neither “tools save tokens” nor “tools waste tokens” but: *the tool is cost-effective exactly where the model cannot do the task without it*. (The output cap right-censors long generations; raw “hit-cap” rates are not comparable across arms, since tool runs are multi-turn and graded from the tool result — a cap-hit on a final narration turn rarely voids the answer, so cap-hit-and-failed stays $\leq 10\%$ in the tool arms. We report totals; a completion-only lens, which flatters the tool, we do not report.)

Robustness: Reasoning Mode and Contamination

Reasoning mode. The same signed rule applied to the *think=on* corpus reproduces the verdict pattern (YES/YES/MIXED/YES, RQ5 YES, RQ6 NO). But the raw *think=on* gaps are budget-confounded: reasoning and answer share one decode budget (6,144 tokens; 8,192 for *solve*), and 14–92% of $\geq 9B$ no-tools *think=on* trials truncate (Gemma-MoE-26B 92%, the 35B only 14%), collapsing the validation baselines (*validate_plan*: 9B 80 $\rightarrow$ 21%, Gemma 88 $\rightarrow$ 10%; *validate_domain* Gemma 78 $\rightarrow$ 0%). We therefore read the cross-mode evidence only through budget-insensitive statistics. The steered tool arm is mode-stable (mean off-vs-on shift 9 pp; the 35B is invariant on all five tasks) while the no-tools baseline swings 28 pp, so essentially all cross-mode movement is a baseline effect, not a change in the tool’s value. The robust floor — the minimum benefit over both modes — keeps *solve* robust (+46–71 pp) and *simulate* sole-source (+83–97 pp; a benefit with no baseline cannot be budget-confounded); the *validate_** tasks are budget-dependent,

What the tokens buy — total tokens/trial vs success, arrows run no-tools (open) to +tool steered (filled) · $\geq 9B$, think=off

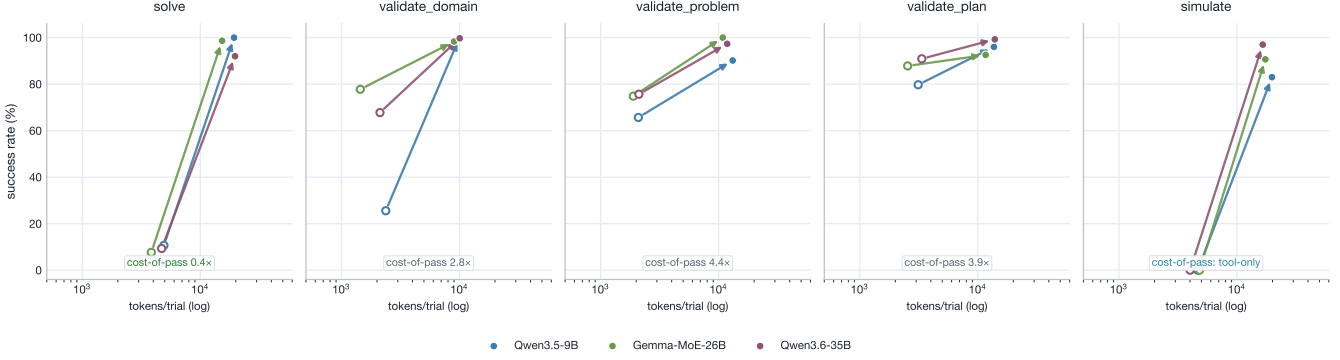


Figure 4: What the tokens buy ($\geq 9B$, *think*=off): mean total tokens per trial (log x) vs. success (y); each arrow runs no-tools→+tool(steered). Every arrow points up-and-right — the tool always costs more per trial; the success it buys is decisive where the baseline is floored (*solve*, *simulate*) and modest where it is already strong (*validate_**).

their large *think*=on gaps being the artifact.

Contamination. Because the domains are public, we re-ran the matrix on structurally identical domains with all surface names renamed (Table 3). The *think*=off no-tools probe is near-null — mean $|\Delta|$ (canonical – anonymized) success 1.1 pp (max 3.7) with zero CI-disjoint task cells — so the unaided baseline carries no detectable memorization signal where it has headroom, and the availability gaps are not a contamination artifact. The per-task breakdown (Table 6) sharpens this: across the 15 $\geq 9B$ (model $\times$ task) cells the mean $|\Delta|$ is 1.2 pp (max 3.7), confirming the null task-by-task, and the largest drifts all favor the *anonymized* corpus — the opposite of the direction memorization of the public domains would produce. The only CI-disjoint cell anywhere is *validate_plan* under *think*=on, and it traces to tokenization, not memory: the renamed prompts run $\approx 5\%$ longer and so truncate more under the shared budget — success falls 4–6 pp in lockstep with completion while success-given-completion is unchanged — itself a demonstration of the budget confound above.

Discussion

Three implications are not specific to PDDL and are natural hypotheses for tool-augmented LLMs more broadly, which our single-formalism, open-weight roster can suggest but not establish. First, the binding constraint on tool-augmented performance here is not whether the model *can* use the tool but whether it *does* — and, the non-obvious part, whether it does is a large, volatile, prompt-sensitive free variable: invocation ranges from 21% to 100% across models on the same task, reverses rank-order between models under reasoning mode, and is moved 21 → 94% by one appended sentence. In nearly every cell accuracy-given-a-call exceeds 92% (9B *simulate*, $\approx 81\%$, is the exception), so the oracle design makes the decomposition $P(\text{call}) \times P(\text{correct} | \text{call})$ exact and localizes almost all of this volatility in the first factor. A correct-by-construction tool turns “can the model plan?” into “will the model delegate?”, and the second ques-

Model	Canonical	Anonymized	Δ
Qwen3.5-0.8B	45.9	45.8	−0.1
Qwen3.5-4B	58.2	56.1	−2.1
Qwen3.5-9B	63.8	64.2	+0.4
Gemma-MoE-26B	74.3	75.2	+0.9
Qwen3.6-35B	75.7	74.8	−0.9

Table 3: Contamination control: no-tools success rate (%), *think*=off, pooled over the five tasks) on the canonical vs. anonymized-domain corpus, per model ($N = 4,560$ trials each side). Every model’s Wilson 95% intervals overlap between corpora (Δ within noise), and across all 25 (model $\times$ task) cells the mean $|\Delta|$ is 1.1 pp (max 3.7) with *zero* CI-disjoint cells — the unaided baseline carries no memorization signal. The null is informative where the baseline has headroom (e.g. Gemma-MoE-26B, Qwen3.6-35B) and uninformative at the floor.

tion is the one that varies — widely. Second, availability and steering are distinct levers with different failure modes: exposing a tool without urging its use leaves a model free to fall back on prose, and on a task it can almost do, that fallback is actively worse than the tool-assisted path it declined. The remedy is cheap — one imperative sentence — but it is not automatic, which is a concrete caution for any system that exposes tools and assumes they will be used. Third, our results are direct empirical support for the LLM-Modulo thesis (Kambhampati et al. 2024) that LLMs should be paired with sound external verifiers, and a refinement of it: the pairing pays off only when the orchestration actually routes work to the verifier, and naive availability does not guarantee that routing. The practical rule that follows is sharp — a planner or validator behind a tool interface helps most when the task exceeds the model’s unaided reach *and* the prompt explicitly directs the call; where the model is already competent and uninstructed, the tool is at best a token-priced luxury and at worst a regression.

Limitations

Several constraints bound these conclusions. Under *think=on*, reasoning and answer share one decode budget, so a large fraction of no-tools trials truncate; we defend the cross-mode claims with budget-insensitive statistics (the robust floor), but cannot fully decouple the tool’s value from the budget in that mode, and a cap-raised rerun is untested. Decoding is deterministic (temperature 0), so each trial is a single sample and our intervals quantify uncertainty over the fixture instances, not sampling variance; robustness to higher temperatures is untested. The roster is five open-weight models from two families — our claims concern the *regime structure* of when a tool helps these models, not a leaderboard, and need not transfer to proprietary or frontier systems. Model scale is moreover partially confounded with numerical precision — the two largest models are served AWQ-INT4 while the smaller three are 16-bit — so the cross-model plain-arm size inversion cannot fully separate spontaneous-adoption propensity from a quantization effect on tool-calling; we therefore frame propensity within-model, through the $P(\text{call}) \times P(\text{correct} \mid \text{call})$ decomposition, which does not rest on that comparison. The availability gap likewise differences two arms whose system prompts, beyond tool presence, also differ by a one-line policy/format clause, so the steering gap — where the plain and steered prompts differ by exactly one appended sentence — is the cleaner of the two contrasts. Grading is strict and end-to-end by design (format adherence is part of the task), so a model that reasons correctly but emits malformed output scores as a failure; we report the failure-type decomposition so this is visible rather than hidden. The serving runtime did not surface per-trial reasoning traces for the model family used, so *think=on* reasoning is evidenced through completion-token counts rather than archived chains of thought; and wall-clock latency is not recoverable from a batched server, so we use token cost, not time, as the efficiency axis. Finally, *validate_domain* carries a 5:1 positive-to-negative imbalance that widens its negative-arm interval.

Future Work

The natural next step is to test whether the regime structure and the availability-versus-steering distinction generalize beyond this suite: to cross-domain planning benchmarks such as PlanBench (Valmeekam et al. 2023) and to a head-to-head comparison with published LLM-as-formalizer baselines (Huang and Zhang 2025), carrying the same end-to-end grading, signed-significance, and contamination controls. Beyond single-tool-use, multi-tool orchestration and tool-chaining — where the model must select among several relevant tools and compose their outputs — would extend the analysis to the agentic regime, and a cap-raised rerun would let the *think=on* benefit be measured without the decode-budget confound.

Conclusion

We asked whether a sound symbolic planner or validator, exposed as a callable tool, improves a language model on individual PDDL tasks. It does, but the benefit is strongly

regime-dependent: the tool is indispensable where the model cannot do the task unaided (solving, state tracking), a token-priced luxury where it is already competent (file validation), and — on plan checking — can even be *harmful* for a model that under-calls it (one of our three $\geq 9\text{B}$ models drops 67 pp when the tool is merely available), a failure one steering sentence reverses. Throughout, the binding constraint is invocation *propensity*, not capability or tool accuracy, and it survives reasoning mode and an anonymized-domain contamination control. The practical rule is concrete: a planner or validator behind a tool interface helps an LLM most when the task exceeds the model’s unaided reach *and* the model is actually prompted to call the tool — availability alone is not enough.

References

- Anthropic. 2024. Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>. Accessed: 2026-06-14.
- Benyamini, Y.; Mordoch, A.; Shperberg, S. S.; and Stern, R. 2025. Toward PDDL Planning Copilot. arXiv:2509.12987.
- Fox, M.; and Long, D. 2003. PDDL2.1: An Extension to PDDL for Expressing Temporal Planning Domains. *Journal of Artificial Intelligence Research*, 20: 61–124.
- Guan, L.; Valmeekam, K.; Sreedharan, S.; and Kambhampati, S. 2023. Leveraging Pre-trained Large Language Models to Construct and Utilize World Models for Model-based Task Planning. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*.
- Helmert, M. 2006. The Fast Downward Planning System. *Journal of Artificial Intelligence Research*, 26: 191–246.
- Hoffmann, J.; and Nebel, B. 2001. The FF Planning System: Fast Plan Generation Through Heuristic Search. *Journal of Artificial Intelligence Research*, 14: 253–302.
- Howey, R.; Long, D.; and Fox, M. 2004. VAL: Automatic Plan Validation, Continuous Effects and Mixed Initiative Planning Using PDDL. In *16th IEEE International Conference on Tools with Artificial Intelligence (ICTAI)*, 294–301.
- Huang, C.; and Zhang, L. 2025. On the Limit of Language Models as Planning Formalizers. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 4880–4904.
- Kambhampati, S.; Valmeekam, K.; Guan, L.; Verma, M.; Stechly, K.; Bhambri, S.; Saldyt, L.; and Murthy, A. B. 2024. Position: LLMs Can’t Plan, But Can Help Planning in LLM-Modulo Frameworks. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 22895–22907.
- Liu, B.; Jiang, Y.; Zhang, X.; Liu, Q.; Zhang, S.; Biswas, J.; and Stone, P. 2023. LLM+P: Empowering Large Language Models with Optimal Planning Proficiency. arXiv:2304.11477.
- McDermott, D.; Ghallab, M.; Howe, A.; Knoblock, C.; Ram, A.; Veloso, M.; Weld, D.; and Wilkins, D. 1998. PDDL—The Planning Domain Definition Language. Technical Report CVC TR-98-003/DCS TR-1165, Yale Center for Computational Vision and Control.

Newcombe, R. G. 1998. Interval Estimation for the Difference Between Independent Proportions: Comparison of Eleven Methods. *Statistics in Medicine*, 17(8): 873–890.

Pallagani, V.; Muppasani, B.; Murugesan, K.; Rossi, F.; Horesh, L.; Srivastava, B.; Fabiano, F.; and Loreggia, A. 2022. Plansformer: Generating Symbolic Plans using Transformers. arXiv:2212.08681.

Patil, S. G.; Mao, H.; Yan, F.; Ji, C. C.-J.; Suresh, V.; Stoica, I.; and Gonzalez, J. E. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 48371–48392.

Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; et al. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *International Conference on Learning Representations (ICLR)*.

Scala, E.; Haslum, P.; Thiébaux, S.; and Ramírez, M. 2016. Interval-Based Relaxation for General Numeric Planning. In *Proceedings of the 22nd European Conference on Artificial Intelligence (ECAI)*, 655–663.

Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*.

Silver, T.; Dan, S.; Srinivas, K.; Tenenbaum, J. B.; Kaelbling, L. P.; and Katz, M. 2024. Generalized Planning in PDDL Domains with Pretrained Large Language Models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 20256–20264.

Tantakoun, M.; Muise, C.; and Zhu, X. 2025. LLMs as Planning Formalizers: A Survey for Leveraging Large Language Models to Construct Automated Planning Models. In *Findings of the Association for Computational Linguistics: ACL 2025*, 25167–25188.

Valmeekam, K.; Marquez, M.; Olmo, A.; Sreedharan, S.; and Kambhampati, S. 2023. PlanBench: An Extensible Benchmark for Evaluating Large Language Models on Planning and Reasoning about Change. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.

Wilson, E. B. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 22(158): 209–212.

Yao, S.; Shinn, N.; Razavi, P.; and Narasimhan, K. 2024. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045.

Technical Appendix

Invocation decomposition. Table 4 reports the mediation behind the Results: strict success factored as $P(\text{call}) \times P(\text{correct} \mid \text{call})$, with $P(\text{correct} \mid \overline{\text{call}})$ shown where a no-call class exists. The between-arm and between-model spread in success lives entirely in $P(\text{call})$; accuracy-given-a-call stays $\geq 92\%$ throughout, except on 9B *simulate* ($\approx 81\%$).

Validate domain per-class accuracy. The fixtures run 5:1 valid-to-invalid, so a constant “VALID” guess scores 83.3% pooled but only 50% balanced. Table 5 shows that unaided, every $\geq 9\text{B}$ model is at or below the pooled trivial line and ranges from near-chance to only modest on the balanced metric, with the tool lifting balanced accuracy to 95–100%. The RQ1 significance test is computed on pooled success, while the balanced-accuracy endpoints we headline are point estimates whose load-bearing negative arm (60 trials) carries the wider interval. Qwen3.5-9B answers “INVALID” almost reflexively (true-positive rate 11.7%, true-negative rate 95.0%), which is why its pooled score (25.6%) and balanced score (53.3%) diverge so sharply.

Per-task contamination. Table 6 disaggregates the pooled contamination null of Table 3, which is dominated by *validate_plan* (3,000 of 4,560 trials). The per-task drifts are uniformly small, and the largest favor the anonymized corpus — the opposite of the direction memorization of the public domains would produce.

Reproducibility Checklist

Instructions for Authors:

This document outlines key aspects for assessing reproducibility. Please provide your input by editing this .tex file directly.

For each question (that applies), replace the “Type your response here” text with your answer.

Example: If a question appears as

```
\question{Proofs of all novel
claims are included}
{(yes/partial/no)}
Type your response here
```

you would change it to:

```
\question{Proofs of all novel
claims are included}
{(yes/partial/no)}
yes
```

Please make sure to:

- Replace **ONLY** the “Type your response here” text and nothing else.
- Use one of the options listed for that question (e.g., **yes**, **no**, **partial**, or **NA**).
- **Not** modify any other part of the \question command or any other lines in this document.

You can \input this .tex file right before \end{document} of your main file or compile it as a stand-alone document. Check the instructions on your conference’s website to see if you will be asked to provide this checklist with your paper or separately.

Model	Task / arm	$P(\text{call})$	$P(\text{c} \text{call})$	$P(\text{c} \overline{\text{call}})$	succ.
Gemma-MoE-26B	<i>val_plan</i> plain	20.7	99.2	0.0	20.6
Gemma-MoE-26B	<i>val_plan</i> steered	93.6	98.9	—	92.6
Qwen3.6-35B	<i>val_plan</i> plain	82.9	99.1	0.0	82.2
Qwen3.6-35B	<i>val_plan</i> steered	99.9	99.4	—	99.3
Qwen3.5-9B	<i>val_plan</i> plain	97.9	95.2	0.0	93.2
Qwen3.6-35B	<i>solve</i> plain	68.0	92.6	0.0	63.0
Qwen3.6-35B	<i>solve</i> steered	93.7	98.2	0.0	92.0

Table 4: Invocation decomposition of strict success (% , $\geq 9\text{B}$, *think=off*). The collapse on plain *val_plan* and its repair under steering are entirely a change in $P(\text{call})$; accuracy-given-a-call ($P(\text{c}|\text{call})$) is $\geq 92\%$ and barely moves between arms. A missing tool call is an ungradeable answer, so $P(\text{c}|\overline{\text{call}}) = 0$ where it applies (“—” marks arms with no no-call class).

Model	no-tools		+tool (steered)	
	pooled	balanced	pooled	balanced
Qwen3.5-9B	25.6	53.3	100.0	100.0
Gemma-MoE-26B	77.8	74.0	98.3	95.0
Qwen3.6-35B	67.8	64.7	99.7	99.2

Table 5: *validate_domain* accuracy ($\geq 9\text{B}$, *think=off*), pooled (majority-positive) vs. balanced (per-class mean). Trivial baselines are 83.3% pooled / 50% balanced. Unaided pooled scores are at or below the trivial line and balanced accuracy is near chance; the tool rescues the task rather than refining a competent baseline.

Model	<i>solve</i>	<i>v_dom</i>	<i>v_prob</i>	<i>v_plan</i>	<i>sim</i>
Qwen3.5-9B	+0.7	+1.1	+3.7	−0.4	+0.0
Gemma-MoE-26B	+0.3	+2.8	+3.0	+0.4	+0.0
Qwen3.6-35B	+0.3	+2.5	+1.0	−1.9	+0.0

Table 6: Per-task contamination control: Δ = anonymized – canonical no-tools success (pp, *think=off*, $\geq 9\text{B}$; same sign convention as Table 3). Mean $|\Delta| = 1.2$ pp, max 3.7, over 15 cells; the largest drifts favor the anonymized corpus.

1. General Paper Structure

- 1.1. Includes a conceptual outline and/or pseudocode description of AI methods introduced (yes/partial/no/NA) **yes**
- 1.2. Clearly delineates statements that are opinions, hypothesis, and speculation from objective facts and results (yes/no) **yes**
- 1.3. Provides well-marked pedagogical references for less-familiar readers to gain background necessary to replicate the paper (yes/no) **yes**

2. Theoretical Contributions

- 2.1. Does this paper make theoretical contributions? (yes/no) **no**

If yes, please address the following points:

- 2.2. All assumptions and restrictions are stated clearly and formally (yes/partial/no) **NA**
- 2.3. All novel claims are stated formally (e.g., in theorem statements) (yes/partial/no) **NA**
- 2.4. Proofs of all novel claims are included (yes/partial/no) **NA**
- 2.5. Proof sketches or intuitions are given for complex and/or novel results (yes/partial/no) **NA**
- 2.6. Appropriate citations to theoretical tools used are given (yes/partial/no) **NA**
- 2.7. All theoretical claims are demonstrated empirically to hold (yes/partial/no/NA) **NA**
- 2.8. All experimental code used to eliminate or disprove claims is included (yes/no/NA) **NA**

3. Dataset Usage

- 3.1. Does this paper rely on one or more datasets? (yes/no) **yes**

If yes, please address the following points:

- 3.2. A motivation is given for why the experiments are conducted on the selected datasets (yes/partial/no/NA) **yes**
- 3.3. All novel datasets introduced in this paper are included in a data appendix (yes/partial/no/NA) **NA**
- 3.4. All novel datasets introduced in this paper will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no/NA) **NA**
- 3.5. All datasets drawn from the existing literature (potentially including authors’ own previously published work) are accompanied by appropriate citations (yes/no/NA) **yes**
- 3.6. All datasets drawn from the existing literature (potentially including authors’ own previously published work) are publicly available (yes/partial/no/NA) **yes**

- 3.7. All datasets that are not publicly available are described in detail, with explanation why publicly available alternatives are not scientifically satisfying (yes/partial/no/NA) **NA**

cal tests (e.g., Wilcoxon signed-rank) (yes/partial/no) **partial**

- 4.13. This paper lists all final (hyper-)parameters used for each model/algorithm in the paper's experiments (yes/partial/no/NA) **yes**

4. Computational Experiments

- 4.1. Does this paper include computational experiments? (yes/no) **yes**

If yes, please address the following points:

- 4.2. This paper states the number and range of values tried per (hyper-) parameter during development of the paper, along with the criterion used for selecting the final parameter setting (yes/partial/no/NA) **partial**
- 4.3. Any code required for pre-processing data is included in the appendix (yes/partial/no) **no**
- 4.4. All source code required for conducting and analyzing the experiments is included in a code appendix (yes/partial/no) **no**
- 4.5. All source code required for conducting and analyzing the experiments will be made publicly available upon publication of the paper with a license that allows free usage for research purposes (yes/partial/no) **yes**
- 4.6. All source code implementing new methods have comments detailing the implementation, with references to the paper where each step comes from (yes/partial/no) **partial**
- 4.7. If an algorithm depends on randomness, then the method used for setting seeds is described in a way sufficient to allow replication of results (yes/partial/no/NA) **NA**
- 4.8. This paper specifies the computing infrastructure used for running experiments (hardware and software), including GPU/CPU models; amount of memory; operating system; names and versions of relevant software libraries and frameworks (yes/partial/no) **partial**
- 4.9. This paper formally describes evaluation metrics used and explains the motivation for choosing these metrics (yes/partial/no) **yes**
- 4.10. This paper states the number of algorithm runs used to compute each reported result (yes/no) **yes**
- 4.11. Analysis of experiments goes beyond single-dimensional summaries of performance (e.g., average; median) to include measures of variation, confidence, or other distributional information (yes/no) **yes**
- 4.12. The significance of any improvement or decrease in performance is judged using appropriate statisti-